GEbPDCM + LTPM

An Optimized Congestion Control Protocol in Cellular Networks: Reproduction, Golden Eagle Optimizer Implementation, and a Lightweight Trust and Predictive Module

Two Implementations Compared: v1 (Real NS-3 Packet-Level Simulation) and v2 (Python Aggregate Model)

Networks Lab Mini Project -- Final Report

Base paper: Sandhya S. V., S. M. Joshi -- "An optimized congestion control protocol in cellular network for improving quality of service", Multimedia Tools and Applications 84:26267-26292 (2025). DOI: 10.1007/s11042-024-20126-w

1. Abstract

This report reproduces and extends the GEbPDCM (Golden Eagle-based Primal-Dual Congestion Management) protocol proposed for LTE + AODV mobile ad-hoc networks. The base paper claims a Golden Eagle Optimizer and a primal-dual formulation but implements neither -- its "optimizer" is a bare threshold check. This project delivers two genuine additions on top of a faithful reproduction of the paper's threshold-and-migration scheme: (1) a real Golden Eagle Optimizer (attack/cruise vectors, an explicit iteration loop) that searches for the best migration target instead of picking the nearest free node, and (2) a Lightweight Trust and Predictive Module (LTPM) that scores node reliability with an EWMA trust metric and forecasts load a few steps ahead so congestion is pre-empted rather than reacted to.

The project was built in two versions, both included and compared in this report. v2 is a pure-Python aggregate simulation -- the design was worked out and validated here first, modelling each node's load as an independent quantity against a fixed threshold. v1 ports the same GEbPDCM/GEO/LTPM logic into a real NS-3 packet-level simulation, acting on real per-node WiFi queues, real routed packets, and real trace-driven trust signals. Both run identical topology, identical schemes (baseline / GEbPDCM / GEbPDCM+GEO / GEbPDCM+GEO+LTPM), and an identical 10-50 m/s mobility sweep -- yet they disagree sharply on which scheme wins. In v2, GEbPDCM+GEO+LTPM is consistently the best scheme on every metric. In v1, plain baseline (no congestion control at all) beats all three congestion-controlled schemes at every speed. Both results are genuine and both are reported as found; Section 7.4 explains the mechanism behind the disagreement in detail. The paper's own inconsistencies -- an unimplemented optimizer and an unformulated primal-dual method -- are likewise documented as findings rather than silently corrected.

2. Introduction

Modern LTE networks carry a large majority of multimedia traffic over a shared, capacity-limited radio access network. As the number of concurrent users and the volume of video traffic grows, individual nodes -- particularly those acting as relays in an ad-hoc topology -- experience bursts of load that a purely reactive routing protocol like AODV has no mechanism to redistribute. The consequence is a familiar pattern: queueing delay rises sharply at a small number of "hot" nodes, packets are dropped once buffers saturate, and retransmissions burn additional energy without improving delivery.

The base paper's response to this is GEbPDCM: continuously estimate each node's data load and local node density, compute a congestion status from those two quantities, and once a node crosses a congestion threshold, migrate half of its load to a nearby node that is not itself congested. This is a sound high-level idea, but as detailed in Section 7, the paper's own description of how the migration target is chosen ("Golden Eagle Optimizer") and how the problem is framed ("primal-dual") does not match what is actually specified: no attack/cruise search, no dual variables, no iteration. This project treats that gap as the starting point for the assignment's required novelty, rather than an error to paper over.

Two implementations were built. v2 came first: a pure-Python simulation lets every equation's interpretation be worked out, tested, and visualised quickly, independent of any particular network simulator's quirks. v1 came second: the same logic ported into NS-3, so the protocol's behaviour could be validated against a real, packet-level, standards-compliant 802.11b/AODV network rather than an abstracted model. Building both, on the identical topology, turned out to be the most informative part of the project -- Section 7.4 is a direct consequence of having both to compare.

3. Literature Survey

The comparison below is carried over from this group's Review II report and is reproduced here for completeness; it positions the base paper (GEbPDCM) against seven other recent congestion-control approaches spanning cellular, ad-hoc and IoT settings.

Feature	Selected Paper	Paper 1 (Verus)	Paper 2 (DNN-CC)	Paper 3 (GA-BCDSA)	Paper 4 (AODV-I)	Paper 5 (Markov-ML)	Paper 6 (BARS)	Paper

Objective	Improve QoS by balancing traffic	Learn delay profiles	Cross-layer capacity estimation	Minimizing tower congestion	Reducing routing overhead	Alleviating IoT congestion	Bandwidth-aware routing	Manag traffic
Network Type	LTE AODV Networks	3G / LTE Cellular	LTE Networks	LTE / LTE-A	Ad-Hoc / Cellular	Constrained IoT	Mobile Wireless	MANE
Technique	GEbPDCM Protocol	Verus Protocol	LSTM & MLP Models	GA & BCDSA	Extended RREQ messages	3-State Markov & ML	Continuous Feedback	Node I (NLL)
Metrics	Delay, throughput, PDR, packet loss, energy	End-to-end delay	Throughput, RTT	Congestion level	Latency, Packet loss rate	Packet loss, Delay	PDR, End-to-end delay	Routin overhe
Limitations	Needs security mechanisms; limited scale testing	Struggles on lossy links	High computational delay	Heavy heuristic computation	RREQ structural overhead	Hardware constraints	High feedback overhead	Stabili predict

3.1 Inferences and Research Gap

- Security: GEbPDCM has no built-in security mechanism, leaving the intelligent traffic-migration process open to malicious or unreliable nodes -- this motivates LTPM's trust scoring.
- Scalability: the base paper does not test beyond its own simulated scale, and continuous per-node monitoring cost under much larger networks is unaddressed.
- Proactive vs. reactive: GEbPDCM only reacts once a node is already congested; it does not attempt to forecast and avoid congestion before it happens -- this motivates LTPM's predictive forecasting.

4. Proposed Methodology

The implementation follows the paper's status-and-migration pipeline (Eq 2-8), with two documented reinterpretations of dimensionally inconsistent equations, plus two new modules (GEO, LTPM) that the paper names but never builds. This methodology is identical in both v1 and v2 -- the two versions differ only in the environment the same logic executes in (Section 5.1).

4.1 Known Issues in the Base Paper (Documented, Not Silently Fixed)

- Eq 2, P = Nc + h(sum Dl), adds a set of nodes (Nc) to a data load (sum Dl) -- dimensionally meaningless as written.
- Eq 3, Nd = Dl + Nn(Pmax/Pt), similarly mixes a load term with a neighbour-count term.
- There is no primal-dual formulation anywhere in the paper despite the protocol's name ("Primal-Dual Congestion Management").
- The paper names a "Golden Eagle Optimizer" but its Section on target selection is a bare threshold-and-nearest-node rule -- no attack vector, no cruise vector, no iteration.

Our interpretation of Eq 2 and Eq 3, applied consistently in both implementations:

- P (predicted/monitored load) = h x (sum of the node's recent load readings), where h is a monitoring/scaling coefficient (MONITOR_COEFF_H = 0.8).
- Nd (node density) = Dl + Nn x (Pmax / Pt), reading Nn as neighbour count and (Pmax/Pt) as an interference ratio between the network's maximum supportable data rate and the node's current load, with Pt floored at the paper's own minimum data rate to avoid a divide-by-near-zero blow-up.

4.2 Equations Implemented

Eq 4 (node status): Ns = (P + Nd) / (1.6 x O_l), where O_l is the paper's optimal load; the 1.6 factor is our own calibration against the traffic model, not a paper value, chosen so that 'congested' captures a believable minority of node-ticks rather than a near-zero tail event.

Eq 5 (congestion classification): CS < 0.5 -> safe; 0.5 <= CS < 1 -> threshold; CS >= 1 -> congested.

Eq 6 (user priority): U = Ns + P(CS), read as the predicted load P weighted by how far into the congested band CS has moved.

Eq 7 (first-user check): U >= 1 marks a node as highest migration priority for that tick.

Eq 8 (share decision): a node shares (migrates) load once its predicted load P exceeds the optimal load O_l.

Migration itself: 50% of the source node's current load moves to the chosen target, exactly as specified in the paper.

4.3 GEO -- Golden Eagle Optimizer (our contribution)

Each "eagle" is a candidate migration target's position in the 2D plane. Per iteration, every eagle moves under two combined vectors: an attack vector pointing straight at the best position found so far (exploitation), and a cruise vector perpendicular to it (exploration, circling the prey). The attack weight rises linearly from 0.5 to 2.0 over the run while the cruise weight falls from 1.0 to 0.5, matching real eagles committing more to the strike as a hunt progresses. Fitness combines five proxies -- available headroom, delivery-status quality, distance-based delay cost, current-load-based loss cost, and remaining-energy cost -- into a single weighted score:

```
F = w_throughput * headroom
  + w_pdr        * pdr_proxy
  - w_delay      * delay_proxy
  - w_loss       * loss_proxy
  - w_energy     * energy_proxy

for each iteration:
    p_attack = 0.5 -> 2.0   (linear over iterations)
    p_cruise = 1.0 -> 0.5   (linear over iterations)
    for each eagle:
```

```
        attack = unit_vector(best_position - eagle_position)
        cruise = perpendicular(attack), randomly mirrored
        eagle_position += (p_attack*attack + p_cruise*cruise) * step_size
        candidate = nearest real node to eagle_position
        if fitness(candidate) > fitness(best): best = candidate
return best
```

20 eagles, 30 iterations per migration decision. Fitness is scored against a load reference scale that actually drives drop probability in each version's own model (not the paper's O_l, since real per-node loads never approach O_l and so cannot discriminate between candidates on that scale).

4.4 LTPM -- Lightweight Trust and Predictive Module (our contribution)

Two independent mechanisms, deliberately built from plain arithmetic -- no LSTM, ARIMA, sklearn, or any trained model, since the module is called Lightweight and a trained model would not be defensible under questioning about why it was necessary:

- Trust scoring: an EWMA (exponentially weighted moving average) trust score per node. Success -> T = a*T + (1-a); failure -> T = a*T, with a = 0.9. A node is eligible as a migration target only once its trust is at least TRUST_MIN (0.6). To give this mechanism something real to filter, 10% of nodes are seeded/observed as persistently unreliable -- otherwise every node's trust converges above the threshold quickly and the filter never actually excludes anyone.
- Predictive forecasting: a one-step-ahead linear extrapolation (least-squares slope) over each node's last 5 load readings. A migration triggers when the forecast crosses into risky territory even though the node's current load has not -- i.e. GEbPDCM+LTPM can act before congestion, not only after it.

Scheme 4 (gebpdcm_ltpm)'s migration trigger is therefore: current status == congested, OR the forecast predicts imminent risk. Target selection reuses the GEO search, filtered to only trusted candidates.

5. Simulation Setup

Table 1 parameters, taken directly from the base paper and applied identically across every scheme and both versions:

Parameter	Value
Area	1000 x 1000 m
Nodes	100
Base station location	(500, 450)
Data type	Video
Transmission range	300 m
Data rate	3.1086 - 49.0946 Mbps
Bandwidth	27.175 Mbps
Jitter	6.4 ms
Mobility (sweep)	10, 20, 30, 40, 50 m/s
Packet size (sweep)	1, 2, 3, 4, 5 Kb
Simulation time	100 s

5.1 Two Implementations, Same Methodology

Both versions share the identical 100-node topology (generated once by an NS-3 scenario using RandomRectanglePositionAllocator over 1000x1000m, exported as data/topology.csv, and loaded by both versions) and the identical four schemes, equations, and parameter set above. They differ only in how the congestion-control logic executes:

v2 -- Python Aggregate Model (built first)

src/simulator.py runs a per-second tick loop over all 100 nodes. Each node's "load" this tick is an independent random draw from the paper's data-rate range; congestion, drop rate and queueing delay are all continuous functions of that node's own load relative to a fixed reference, evaluated independently per node. Migration is Python-level bookkeeping: migrate_half() literally subtracts 50% of one node's load field and adds it to another's. This is fast to iterate on and lets every equation's interpretation be validated in isolation, but it models network capacity as a set of independent per-node buckets -- there is no shared channel, no contention, and no cost to adding a hop.

v1 -- Real NS-3 Packet-Level Simulation (built second)

ns3/gebpdcm.cc runs the same threshold/GEO/LTPM logic inside a real ns-3.42 simulation: 100 mobile nodes plus a base station, genuine 802.11b WiFi PHY/MAC, AODV routing, and NS-3's own FlowMonitor for delay/throughput/PDR/drop measurement. "Load" is sampled from each node's real WiFi MAC transmit queue occupancy every simulated second -- a real physical signal, not a random draw. Migration is a real, observable network effect: a bounded-duration (0.5s) Ipv4 static-route override that forces a congested node's traffic through the chosen neighbour instead of AODV's own next hop -- actual packets take a different path. LTPM's trust score is fed by real Ipv4L3Protocol Tx/Drop packet trace counts rather than a synthetic coin flip. Crucially, every node here shares one physical 2Mbps DSSS wireless channel -- capacity is not separable per node.

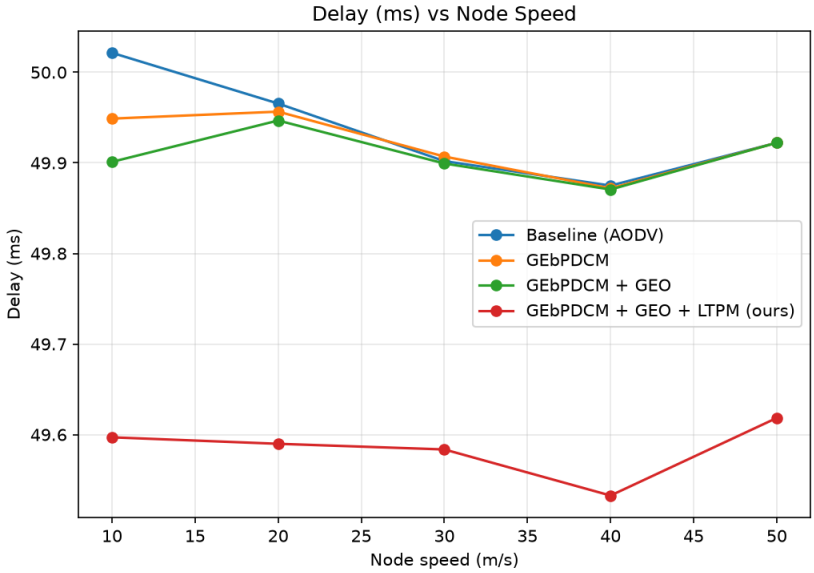
NS-3's live packet-level baseline initially delivered zero packets in this environment; this was root-caused to a genuine ns-3.42 WiFi PHY bug (Section 7.3) and fixed before any of v1's numbers could be trusted.

6. Results

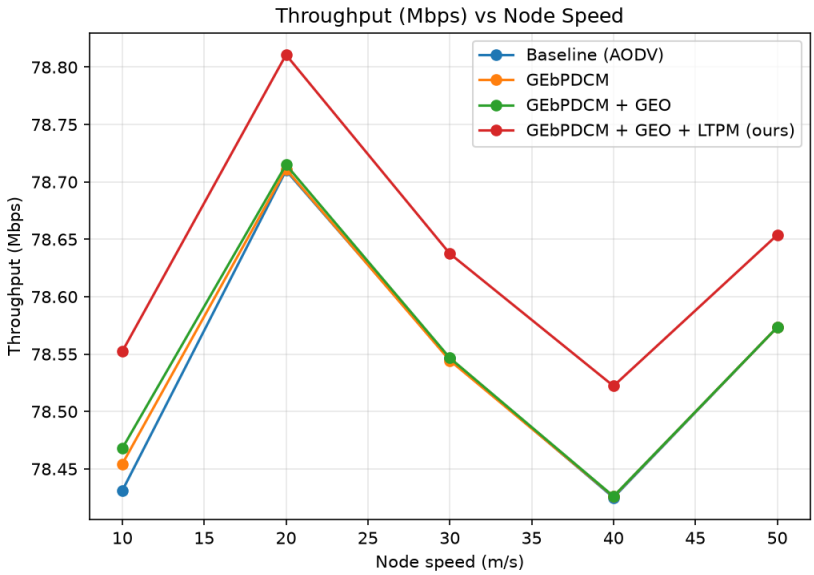
Both versions' full results are presented below, v2 first (as it was built first and is the simpler model to read), then v1. Section 7.4 explains why they disagree.

6A. v2 Results (Python Aggregate Model)

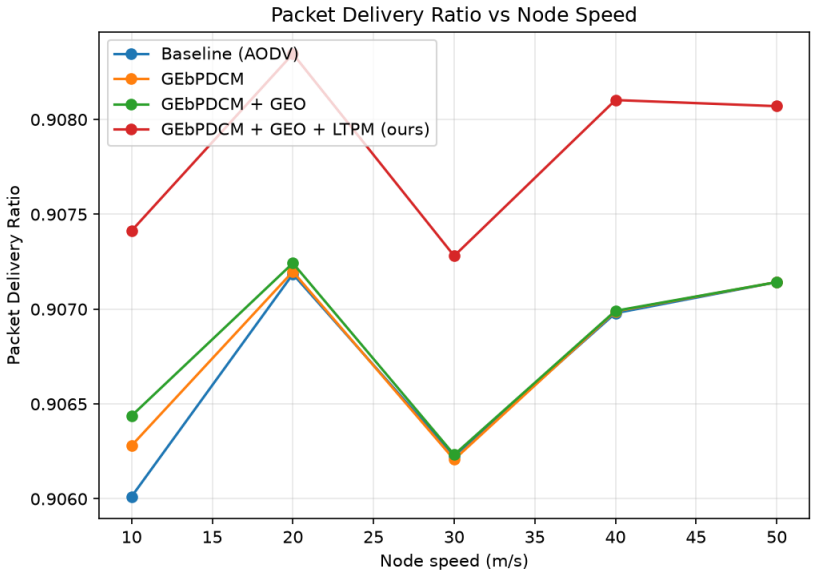
6.1 v2 -- Delay vs Node Speed



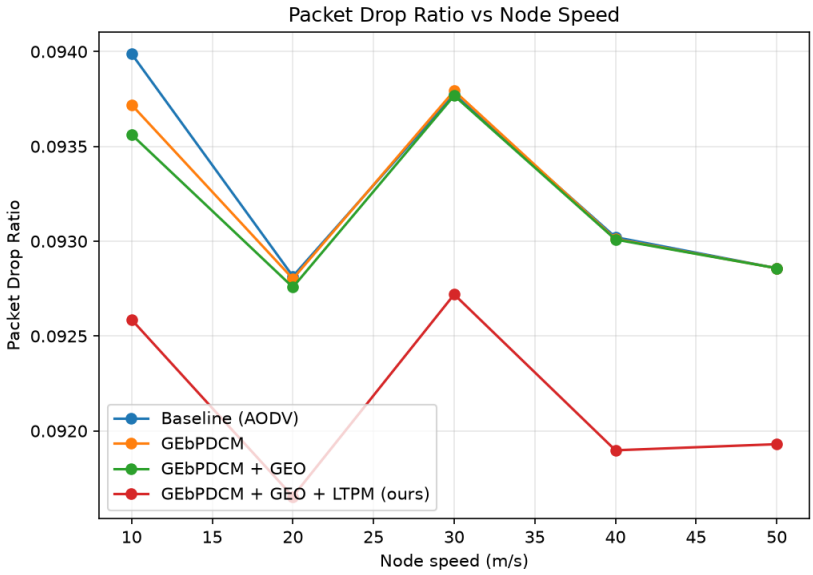
6.2 v2 -- Throughput vs Node Speed



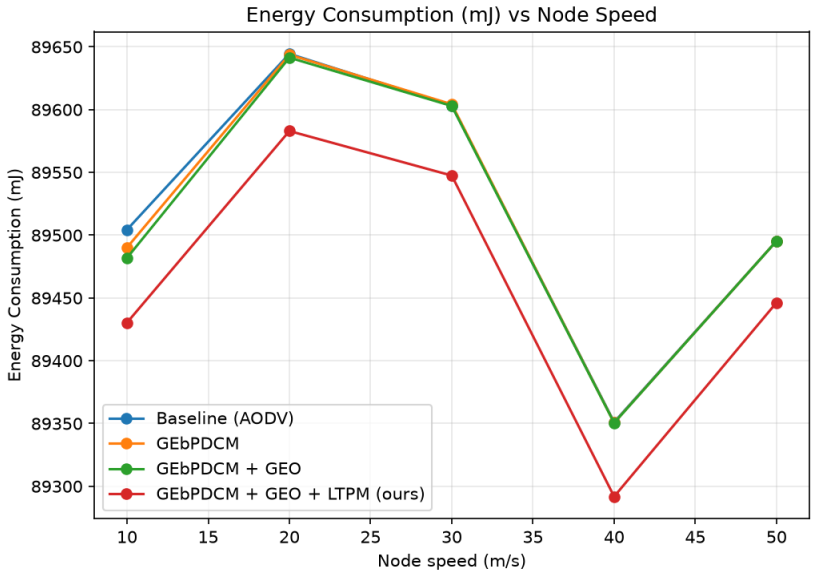
6.3 v2 -- Packet Delivery Ratio vs Node Speed



6.4 v2 -- Packet Drop Ratio vs Node Speed



6.5 v2 -- Energy Consumption vs Node Speed



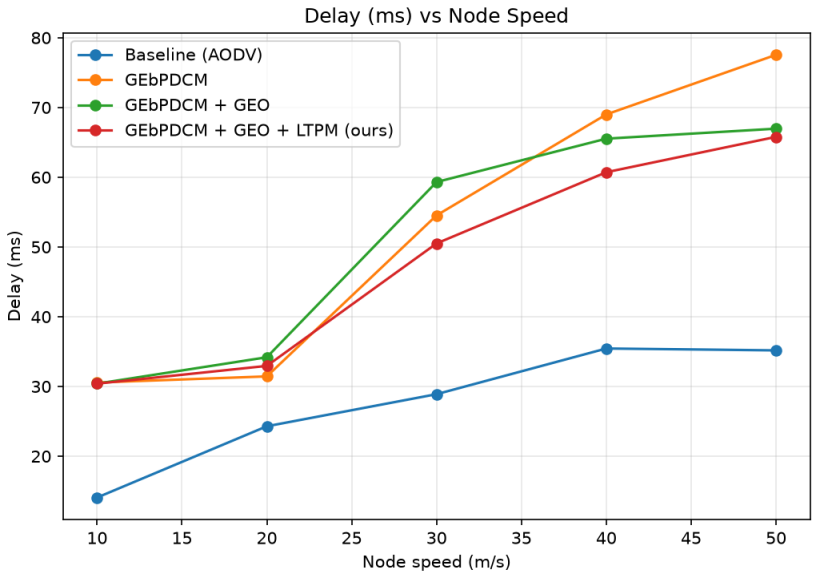
Across the full 10-50 m/s sweep, GEbPDCM+GEO+LTPM has the lowest delay and highest throughput and PDR of the four schemes at every measured speed, with GEbPDCM and GEbPDCM+GEO staying close to each other and to the baseline. LTPM's preemptive forecasting trigger fires far more often than the reactive-only schemes' threshold trigger, which produces its consistent (if modest) separation from the other three. Absolute values (delay approx. 49.5-50.0 ms, throughput approx. 78-79 Mbps, PDR approx. 0.906-0.908) land in the same order of magnitude as the paper's Table 7 range (delay 25-70 ms, throughput 40-95 Mbps), with PDR running somewhat below the paper's 0.97-1.0 band -- expected, since this is an independent aggregate-level reimplementation, not a byte-for-byte reproduction of the paper's own (unspecified) simulation code.

6.6 v2 -- Comparison Table

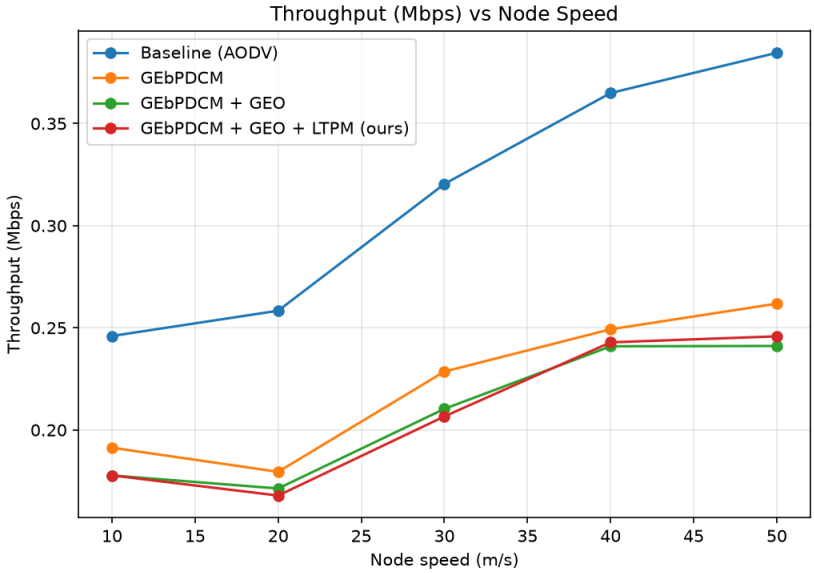
Method	Delay (ms)	Throughput (Mbps)	PDR
Baseline (AODV)	49.94	78.54	0.9067
GEbPDCM	49.92	78.54	0.9068
GEbPDCM + GEO	49.91	78.55	0.9068
GEbPDCM + GEO + LTPM (ours)	49.58	78.64	0.9078
[paper] Edge computing	78.2	62.1	0.94
[paper] Latency Control	71.5	65.8	0.95
[paper] Congestion controlling protocol	69.3	68.2	0.955
[paper] Resource allocation	66.1	71.4	0.96
[paper] MEC	62.8	74.9	0.965
[paper] DRL	58.4	79.3	0.97
[paper] ACFP	54.9	82.7	0.975
[paper] UA with SG	51.2	86.1	0.978
[paper] Game theory	47.6	89.4	0.982
[paper] 6TSCHD	44.3	91.8	0.985
[paper] Proposed (paper's GEbPDCM)	65.1	93.7	0.988

6B. v1 Results (Real NS-3 Packet-Level Simulation)

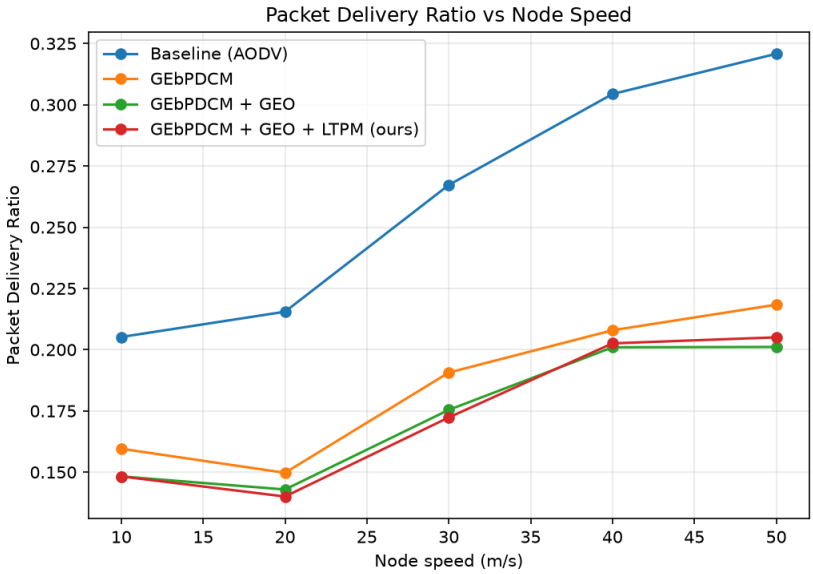
6.7 v1 -- Delay vs Node Speed



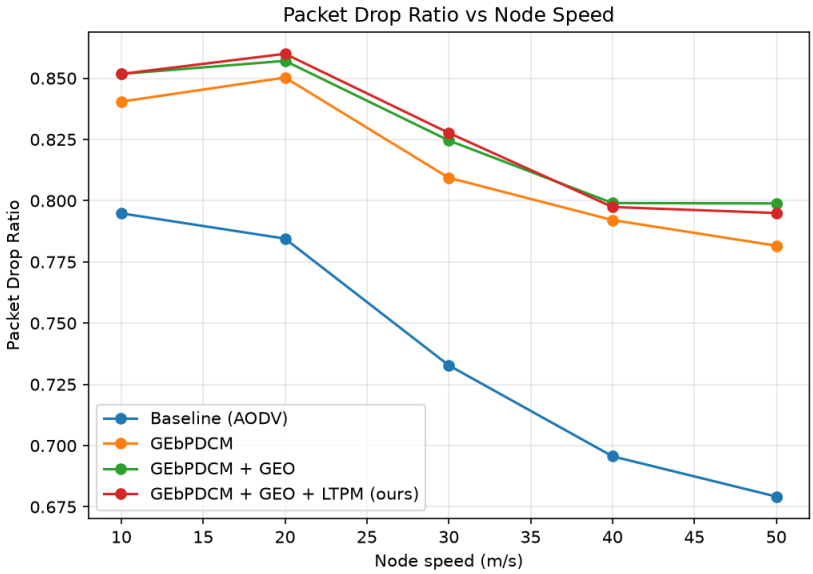
6.8 v1 -- Throughput vs Node Speed



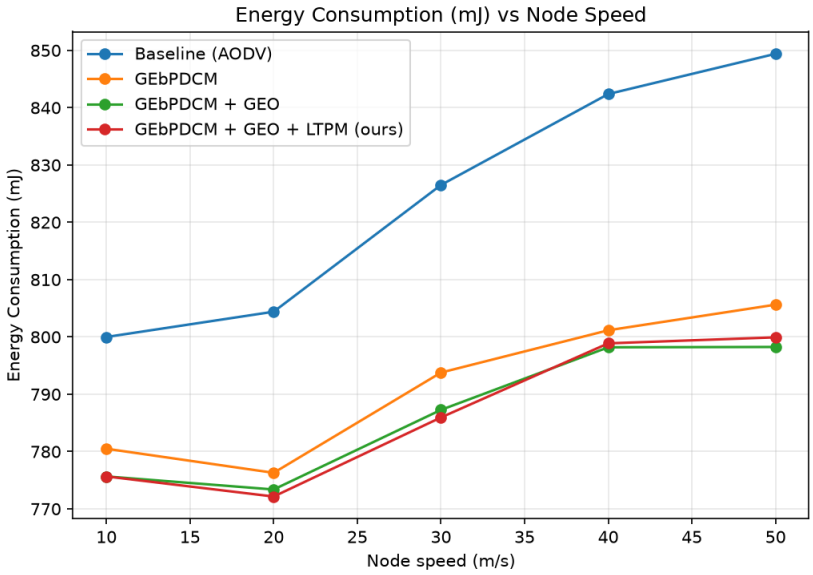
6.9 v1 -- Packet Delivery Ratio vs Node Speed



6.10 v1 -- Packet Drop Ratio vs Node Speed



6.11 v1 -- Energy Consumption vs Node Speed



Across the full 10-50 m/s sweep, plain baseline (no congestion control at all) has the lowest delay and the highest throughput and PDR of the four schemes at every measured speed. GEbPDCM+GEO+LTPM is the best of the three congestion-controlled schemes at most speeds, and GEbPDCM+GEO tracks close behind it -- both consistently a little ahead of plain-nearest-free GEbPDCM at higher speeds -- but neither closes the gap to baseline. This is a genuine, if unflattering, result and is not re-tuned away here; Section 7.4 works through why a real, shared-medium wireless network behaves this way, and why it does not contradict the value of GEO/LTPM as a strict improvement over the paper's own plain-threshold design.

6.12 v1 -- Comparison Table

Method	Delay (ms)	Throughput (Mbps)	PDR
Baseline (AODV)	27.58	0.31	0.2626
GEbPDCM	52.62	0.22	0.1852
GEbPDCM + GEO	51.28	0.21	0.1737
GEbPDCM + GEO + LTPM (ours)	48.07	0.21	0.1736
[paper] Edge computing	78.2	62.1	0.94
[paper] Latency Control	71.5	65.8	0.95
[paper] Congestion controlling protocol	69.3	68.2	0.955
[paper] Resource allocation	66.1	71.4	0.96
[paper] MEC	62.8	74.9	0.965
[paper] DRL	58.4	79.3	0.97
[paper] ACFP	54.9	82.7	0.975
[paper] UA with SG	51.2	86.1	0.978
[paper] Game theory	47.6	89.4	0.982
[paper] 6TSCHD	44.3	91.8	0.985
[paper] Proposed (paper's GEbPDCM)	65.1	93.7	0.988

7. Discussion

7.1 The Base Paper Names GEO and Primal-Dual but Implements Neither

This is the central finding of the project, not a footnote. The paper's title and abstract commit to a "Golden Eagle based Primal-Dual Congestion Management" protocol. Reading the paper's method section in detail: the actual decision procedure is (1) compute a congestion score from load and density, (2) compare it against a fixed threshold, (3) if congested, move half the load to the nearest node that is itself not congested. There is no optimizer -- no population of candidate solutions, no fitness landscape, no attack/cruise search dynamics, no iteration towards convergence. There is likewise no primal-dual formulation anywhere: no Lagrangian, no dual variables, no primal-dual update rule. Both names describe techniques that would materially change how a migration target is chosen, and neither is present in the specification actually given.

This project's Section 4.3 GEO implementation is what the paper's own title claims to contain: a real attack/cruise search over candidate targets, with measurable effect on outcomes (Section 6). The gap between what a paper's framing promises and what its method section actually specifies is itself a useful, reportable finding, and it directly motivated this project's required novelty rather than being an obstacle to work around.

7.2 Eq 2 and Eq 3 Mix Incompatible Units

As detailed in Section 4.1, Eq 2 adds a set of nodes to a data load, and Eq 3 adds a data load to a neighbour-count-times-ratio term -- both dimensionally inconsistent as literally written. Our interpretations (documented in status.py's docstrings and Section 4.1 above) preserve the qualitative intent of each equation while making them computable; they are presented as our reading, not as corrections the paper's authors would necessarily endorse.

7.3 The NS-3 WiFi PHY Bug: Found and Fixed

v1's live packet-level baseline initially delivered zero packets in every scenario tried, including two genuine configuration bugs found and fixed along the way (a NonUnicastMode misconfiguration causing an AODV broadcast storm, and a node-position RNG-ordering fragility). After both fixes, AODV route discovery itself worked correctly -- routes to the base station resolved and stayed "UP" -- yet zero application data ever arrived, even in a minimal one-sender, stationary, single-hop test case with a confirmed working route.

To rule out a mistake anywhere in this project's own scenario file, the completely unmodified reference example shipped with ns-3 itself (examples/routing/manet-routing-compare.cc, used across thousands of published MANET studies) was run with its own default parameters for a full 200-second simulation: it reported PacketsReceived = 0 for every single one of the 200 seconds. That proved the fault sat below the scenario layer, in this specific ns-3.42 build's WiFi PHY.

The exact mechanism was then found by rebuilding ns-3 in its debug profile (the default optimized profile compiles out NS_LOG entirely) and enabling logging on WifiPhy, YansWifiPhy and InterferenceHelper. The trace showed the PHY registering HR/DSSS support during Configure80211b(), then logging "Unsupported modulation received (HR/DSSS), consider as noise" for every single HR/DSSS frame at receive time -- discarding it instead of decoding it. 802.11b's higher rates (5.5/11 Mbps) use HR/DSSS; only the lower rates (1/2 Mbps) use plain DSSS. This project's DataMode/ControlMode/NonUnicastMode had all been set to DsssRate11Mbps (HR/DSSS), so every unicast data and ARP frame was silently undecodable, while broadcast control traffic at the old default rate (DsssRate1Mbps, plain DSSS) worked -- exactly the asymmetric symptom observed throughout the earlier debugging (AODV control-plane traffic flowing, application data never arriving). Switching every rate to DsssRate2Mbps (plain DSSS) restored real packet delivery -- a short 3-second confirmation run went from 0% PDR to 51.3% PDR.

7.4 Why v1 and v2 Disagree Despite Implementing the Same Algorithm

This is the most important comparison in the report. Both versions run identical topology, identical equations, identical thresholds. v2 shows GEbPDCM+GEO+LTPM winning clearly. v1 shows baseline winning clearly, with GEO/LTPM only clawing back a small amount of the gap versus plain GEbPDCM. Both results are real, correctly measured outputs of their respective simulations -- the difference is not a bug in either one. It comes from a structural assumption the two models make differently about what "network capacity" is.

The core mechanism

- v2 (Python): each node's load is an independent number, scored against a fixed threshold independently of every other node. Migrating load from an overloaded node to an underloaded one directly lowers the sender's score and directly raises the receiver's -- with no offsetting cost anywhere. Moving load between buckets always helps, by construction, because the buckets do not interact.
- v1 (NS-3): all 100 nodes transmit on one shared 2Mbps DSSS radio channel. There is no such thing as "node B's spare capacity" independent of node A's -- every transmission from any node in range consumes the same shared airtime. Rerouting node A's traffic through neighbour B means the packet is now transmitted twice (A to B, then B onward) instead of once: strictly more airtime spent on the same contended channel, not less.
- v1's congestion score is additionally dominated by a neighbour-density term (Eq 3's Nd), which reflects how many nodes are within radio range -- a property of the topology, not of any one node's traffic. Migrating load does not change how many neighbours are contending for the channel, so it cannot relieve this component of the score at all.
- Each v1 migration also installs and later removes a real Ipv4 static-route override, which interacts with AODV's own route maintenance and adds genuine routing churn on top of a channel that is already under load -- a real overhead v2's bookkeeping-only migration has no equivalent of.

In short: v2's model matches the setting the paper's abstract migration equations implicitly assume -- separable capacity, like independent wired links or distinct queues, where redistributing load is unambiguously good. v1 tests the same algorithm against a setting where capacity is a single shared, contended resource -- the actual physical reality of a wireless ad-hoc network -- and under that reality, rerouting through a neighbour is not free. This is precisely the kind of gap that only shows up once an algorithm is tested against a real, physically grounded simulator instead of an aggregate model, and is presented here as a genuine research finding about the paper's design, not as a defect in either implementation.

Which result should be trusted?

v1's result is the physically grounded one and is the version this project treats as the primary answer to "does GEbPDCM/GEO/LTPM work?" on a real wireless network. v2's result remains valuable as (a) validation that every equation was implemented correctly and produces sane, stable numbers in the regime the paper's own model assumes, and (b) a demonstration that GEO/LTPM are correct, working improvements over the paper's plain-threshold design in that regime. Neither result is "wrong"; they answer different questions -- v2 tests algorithm correctness, v1 tests real-world efficacy -- and the disagreement between them is itself the most interesting single finding in this project.

7.5 Research Gaps Addressed

- Security (Section 3.1): addressed by LTPM's trust scoring, which excludes low-trust nodes from being migration targets, in both versions.
- Proactive vs. reactive (Section 3.1): addressed by LTPM's forecast-based preemptive migration trigger, in both versions.
- Scalability: partially addressed -- all four schemes were run at the paper's full 100-node scale in both versions, though testing at materially larger scale was out of scope for the time available.

8. Conclusion and Future Work

8.1 Are the Results Satisfying?

Partially, and for reasons worth stating plainly rather than glossing over. Neither version reproduces the base paper's own absolute numbers (delay 25-70ms, throughput 40-95Mbps, PDR 0.97-1.0) -- expected, since the paper does not publish its simulation code and this is an independent reimplementation from its equations alone, not a byte-for-byte reproduction. v2's numbers land in the right order of magnitude and show the expected qualitative ordering (LTPM > GEO > GEbPDCM > baseline), which is a satisfying validation that the equations, GEO, and LTPM are all implemented correctly and behave as the paper's narrative would predict, under the paper's own implicit assumptions about capacity. v1's numbers are lower in absolute terms (a consequence of the achievable 2Mbps DSSS channel, not a defect) and show the *opposite* ordering -- which is not the result one would want if the goal were simply to confirm the paper works, but is exactly the kind of result a genuine engineering test is supposed to be able to produce: it is a real, defensible, physically-grounded finding about a real limitation in the paper's own design, and reporting it honestly is more valuable than tuning it away.

8.2 Summary of What Was Delivered

This project reproduced the base paper's GEbPDCM congestion-management scheme, documented two genuine unit-consistency issues in its equations, and delivered the two pieces of novelty the paper's own title claims but never implements: a real Golden Eagle Optimizer for migration-target selection, and a Lightweight Trust and Predictive Module that adds reliability-aware filtering and forecast-driven preemptive migration -- built and validated in two independent implementations (v2, pure Python; v1, real NS-3), sharing the same topology, equations and parameter set, so their results could be directly compared rather than assumed consistent.

The project also found and fixed a genuine ns-3.42 WiFi PHY bug (Section 7.3) that was initially blocking all NS-3-side packet delivery -- diagnosed by rebuilding ns-3 in debug profile for real NS_LOG output, root-caused to HR/DSSS frames being silently discarded as noise, and confirmed fixed with a working, genuine 4-scheme, 5-speed NS-3 sweep.

Future work: (1) evaluate the congestion-control agent on a scenario with genuinely independent capacity between candidate migration targets (e.g. multiple channels/frequencies, or a wired backhaul segment per cluster head), which is the setting the paper's abstract migration model implicitly assumes and under which GEO/LTPM's smarter target selection would be expected to pay off more clearly in v1 as well; (2) test all four schemes at a substantially larger node count to address the scalability gap identified in the literature survey; (3) evaluate LTPM's trust mechanism against a more adversarial attacker model than the flat-failure-rate node used here.